Mid Term Exam

CPCS-223

Thursday October 12, 2023

Question # 01

Part - A

Write down at least 5 characteristics of a good algorithms

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics
1	It accepts one or more inputs

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics
1	It accepts one or more inputs
2	It returns at least one output

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics
1	It accepts one or more inputs
2	It returns at least one output
3	It terminates after finite steps

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics
1	It accepts one or more inputs
2	It returns at least one output
3	It terminates after finite steps
4	It is independent of a specific programming language, machine or compiler

Question # 01

Write down at least 5 characteristics of a good algorithms

S. No.	Characteristics
1	It accepts one or more inputs
2	It returns at least one output
3	It terminates after finite steps
4	It is independent of a specific programming language, machine or compiler
5	Unlike the computer programs (which are for computers understanding), algorithms are understood by humans.

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

$$\gcd(31415, 14142)$$

Euclid's algorithm for computing $\gcd(m, n)$

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$\begin{aligned} &\gcd(31415, 14142) \\ &= \gcd(14142, 3131) \end{aligned}$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

$$\begin{aligned} &\gcd(31415, 14142) \\ &= \gcd(14142, 3131) \\ &= \gcd(3131, 1618) \end{aligned}$$

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$\begin{aligned} &\gcd(31415, 14142) \\ &= \gcd(14142, 3131) \\ &= \gcd(3131, 1618) \\ &= \gcd(1618, 1513) \end{aligned}$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$\begin{aligned} &\gcd(31415, 14142) \\ &= \gcd(14142, 3131) \\ &= \gcd(3131, 1618) \\ &= \gcd(1618, 1513) \\ &= \gcd(1513, 105) \end{aligned}$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142) \\
&= \gcd(14142, 3131) \\
&= \gcd(3131, 1618) \\
&= \gcd(1618, 1513) \\
&= \gcd(1513, 105) \\
&= \gcd(105, 43)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)\\
&= \gcd(19, 5)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)\\
&= \gcd(19, 5)\\
&= \gcd(5, 4)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)\\
&= \gcd(19, 5)\\
&= \gcd(5, 4)\\
&= \gcd(4, 1)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)\\
&= \gcd(19, 5)\\
&= \gcd(5, 4)\\
&= \gcd(4, 1)\\
&= \gcd(1, 0)
\end{aligned}
$$

Question # 01

Part - B

Find gcd **(31415, 14142)** by applying Euclid's algorithm given below. You need to show all recursive calls with respective values of the parameters.

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

$$
\begin{aligned}
&\gcd(31415, 14142)\\
&= \gcd(14142, 3131)\\
&= \gcd(3131, 1618)\\
&= \gcd(1618, 1513)\\
&= \gcd(1513, 105)\\
&= \gcd(105, 43)\\
&= \gcd(43, 19)\\
&= \gcd(19, 5)\\
&= \gcd(5, 4)\\
&= \gcd(4, 1)\\
&= \gcd(1, 0)\\
&= 1
\end{aligned}
$$

Question # 02

Write an algorithm / pseudocode or code segment using **"Brute Force"** technique to find **number of terms generated** for this sequence by reaching to the conjecture i. e. **term become 1**

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {



}
```

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

    int count = 1;









}
```

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

        int count = 1;
        while (n != 1){




}
```

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

        int count = 1;
        while (n != 1){
            if (n%2 == 0)




}
```

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

        int count = 1;
        while (n != 1){
            if (n%2 == 0)
                n = n/2;



}
```

Question # 02
Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

        int count = 1;
        while (n != 1){
            if (n%2 == 0)
                n = n/2;
            else
                n = n*3 + 1;



}
```

Question # 02

Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {

        int count = 1;
        while (n != 1){
            if (n%2 == 0)
                n = n/2;
            else
                n = n*3 + 1;
            count++;
        }


}
```

Question # 02

Write an algorithm / pseudocode or code segment using "Brute Force" technique to find number of terms generated for this sequence by reaching to the conjecture i. e. term become 1

The Collatz conjecture is one of the most famous unsolved problems in mathematics. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. It concerns sequences of integers in which each term is obtained from the previous term as follows: if the previous term is even, the next term is one half of the previous term. If the previous term is odd, the next term is 3 times the previous term plus 1. The conjecture is that these sequences always reach 1, no matter which positive integer is chosen to start the sequence.

```
public int collatConj(int n) {
        int count = 1;
        while (n != 1){
            if (n%2 == 0)
                n = n/2;
            else
                n = n*3 + 1;
            count++;
        }
        return count;
}
```

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

   Ω can hold only if c<= 0 i.e. "c" is negative which contradicts the basic condition (c is positive constant). Hence proved the expression.

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

   Ω can hold only if c<= 0 i.e. "c" is negative which contradicts the basic condition (c is positive constant). Hence proved the expression.

4. $2^{n+1} = O(2^n)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

   Ω can hold only if c<= 0 i.e. "c" is negative which contradicts the basic condition (c is positive constant). Hence proved the expression.

4. $2^{n+1} = O(2^n)$

   $n_0 = 1, c >= 2$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

   Ω can hold only if c<= 0 i.e. "c" is negative which contradicts the basic condition (c is positive constant). Hence proved the expression.

4. $2^{n+1} = O(2^n)$

   $n_0 = 1, c >= 2$

5. $2.n^2 + 3.n + 6 = \Theta(n^3)$

Question # 03

Prove / Disprove the followings:

$5.n^2 + 10 = \omega(n)$

$2n^2 = o(n^3)$

$100.n + 5 \neq \Omega(n^2)$

$2^{n+1} = O(2^n)$

$2.n^2 + 3.n + 6 = \Theta(n^3)$

1. $5.n^2 + 10 = \omega(n)$

   $n_0 = 1, c < 15$

2. $2n^2 = o(n^3)$

   $n_0 = 1, c > 2$

3. $100.n + 5 \neq \Omega(n^2)$

   Ω can hold only if c<= 0 i.e. "c" is negative which contradicts the basic condition (c is positive constant). Hence proved the expression.

4. $2^{n+1} = O(2^n)$

   $n_0 = 1, c >= 2$

5. $2.n^2 + 3.n + 6 = \Theta(n^3)$

   not true for Ω

   holds for O for $n_0 = 1, c >= 11$

Question # 04

Using the **"Recursion Tree Method"** find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Question # 04

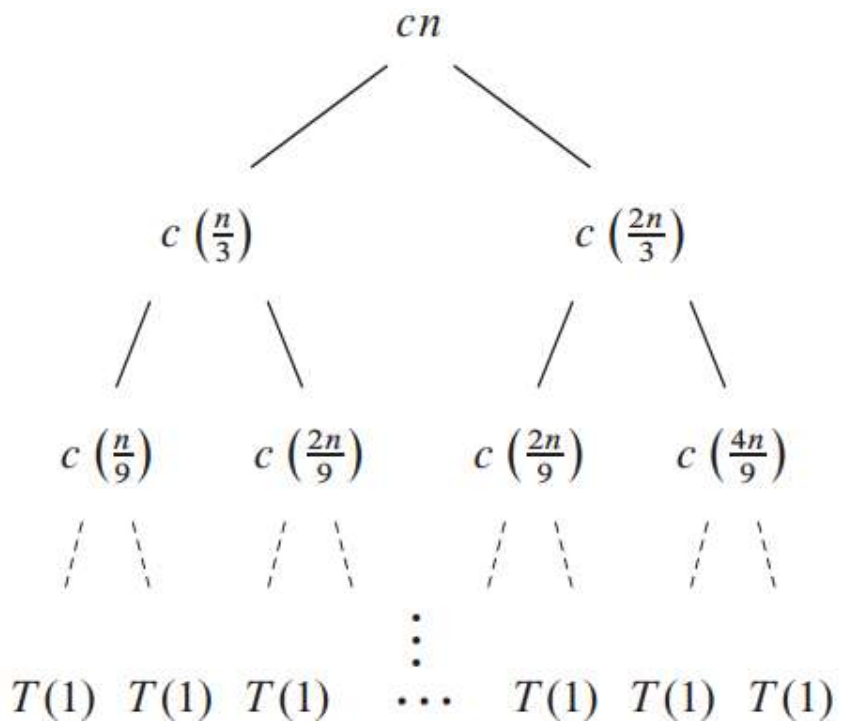
Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Recursion Tree

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Recursion Tree



Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i" $(2/3)^i n$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i n$
Cost of node at level "i"	$c\, n\, (2/3)^i$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i$ n
Cost of node at level "i"	c n $(2/3)^i$
No. of nodes at level "i"	1

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i n$
Cost of node at level "i"	$c\, n\, (2/3)^i$
No. of nodes at level "i"	1
Cost of level "i"	$c\, n\, (2/3)^i$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i n$
Cost of node at level "i"	$c\, n\, (2/3)^i$
No. of nodes at level "i"	1
Cost of level "i"	$c\, n\, (2/3)^i$
Last Level number	$\log_{3/2} n$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i n$
Cost of node at level "i"	$c\, n\, (2/3)^i$
No. of nodes at level "i"	1
Cost of level "i"	$c\, n\, (2/3)^i$
Last Level number	$\log_{3/2} n$
No. of nodes at last Level	1

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Calculations

Problem Size at level "i"	$(2/3)^i n$
Cost of node at level "i"	$c\, n\, (2/3)^i$
No. of nodes at level "i"	1
Cost of level "i"	$c\, n\, (2/3)^i$
Last Level number	$\log_{3/2} n$
No. of nodes at last Level	1
Cost of last Level	$\Theta(1)$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$T(n) = cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1)$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$T(n) = cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1)$$

$$= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1)$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$
\begin{aligned}
T(n) &= cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1) \\
&= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{(2/3)^{\log_{3/2} n} - 1}{(2/3) - 1} cn + \Theta(1)
\end{aligned}
$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$
\begin{aligned}
T(n) &= cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1) \\
&= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{(2/3)^{\log_{3/2} n} - 1}{(2/3) - 1} cn + \Theta(1) \\
&< \sum_{i=0}^{\infty} \left(\frac{2}{3}\right)^i cn + \Theta(1)
\end{aligned}
$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$
\begin{aligned}
T(n) &= cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1) \\
&= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{(2/3)^{\log_{3/2} n} - 1}{(2/3) - 1} cn + \Theta(1) \\
&< \sum_{i=0}^{\infty} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{1}{1 - (2/3)} cn + \Theta(1)
\end{aligned}
$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$
\begin{aligned}
T(n) &= cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1) \\
&= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{(2/3)^{\log_{3/2} n} - 1}{(2/3) - 1} cn + \Theta(1) \\
&< \sum_{i=0}^{\infty} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{1}{1 - (2/3)} cn + \Theta(1) \\
&= 3\,cn + \Theta(1)
\end{aligned}
$$

Question # 04

Using the "Recursion Tree Method" find the complexity class for the given algorithm. Provide the solution for maximum or minimum running time for the given recurrence.

Running Time

$$
\begin{aligned}
T(n) &= cn + \frac{2}{3}cn + \left(\frac{2}{3}\right)^2 cn + \cdots + \left(\frac{2}{3}\right)^{\log_{3/2} n - 1} cn + \Theta(1) \\
&= \sum_{i=0}^{\log_{3/2} n - 1} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{(2/3)^{\log_{3/2} n} - 1}{(2/3) - 1} cn + \Theta(1) \\
&< \sum_{i=0}^{\infty} \left(\frac{2}{3}\right)^i cn + \Theta(1) \\
&= \frac{1}{1 - (2/3)} cn + \Theta(1) \\
&= 3\,cn + \Theta(1) \\
&= O(n) .
\end{aligned}
$$

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

- This algorithm returns "True" if its input matrix is **symmetric** and "False" if it is not.

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

- The basic operation performed is the **comparison** of two matrix elements.

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Note: for part "c" use the RAM model

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Note: for part "c" use the RAM model

Consider the following algorithm.

	Line	Cost	Times
ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$	1	C1	$\sum_{i=0}^{n-2} 1$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers			
for $i \leftarrow 0$ **to** $n-2$ **do**			
for $j \leftarrow i+1$ **to** $n-1$ **do**	2	C2	$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$
if $A[i, j] \neq A[j, i]$			
return false	3	C3	1
return true			

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1)$$

$$= \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$= (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \quad \text{Since } \sum_{i=0}^{n} i = \frac{n(n+1)}{2}$$

$$= (n-1)(n-1) - \frac{(n-2)(n-1)}{2} \quad \text{Since } \sum_{i=0}^{n-2} 1 = n-1$$

$$= n^2 - 2n + 1 - \frac{(n^2 - 3n + 2)}{2}$$

$$= \frac{2n^2 - 4n + 2 - n^2 + 3n - 2}{2} \quad = \frac{(n-1)n}{2}. \quad = \theta(n^2)$$

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
```
//Input: A matrix A[0..n − 1, 0..n − 1] of real numbers
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i, j] ≠ A[j, i]
            return false
return true
```

Question # 05

What does this algorithm compute?

What is its basic operation?

How many times is the basic operation executed?

What is the efficiency class of this algorithm?

Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$
//Input: A matrix $A[0..n-1, 0..n-1]$ of real numbers
for $i \leftarrow 0$ **to** $n-2$ **do**
  for $j \leftarrow i+1$ **to** $n-1$ **do**
    if $A[i, j] \neq A[j, i]$
      return false
return true

- The time efficiency of this algorithm is $\Theta(n^2)$.

Question # 06

Solve the following recurrence relations

$$x(n) = 3x(n-1)$$

for $n > 1$, $x(1) = 4$

$$x(n) = x(n/2) + n$$

for $n > 1$, $x(1) = 1$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$$

$$x(n) \quad = \quad 3x(n-1)$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] = 3^2 x(n-2) \end{aligned}$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] = 3^2 x(n-2) \\ &= 3^2[3x(n-3)] = 3^3 x(n-3) \end{aligned}$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] = 3^2 x(n-2) \\ &= 3^2[3x(n-3)] = 3^3 x(n-3) \\ &= \ldots \\ &= 3^i x(n-i) \end{aligned}$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] = 3^2 x(n-2) \\ &= 3^2[3x(n-3)] = 3^3 x(n-3) \\ &= \ldots \\ &= 3^i x(n-i) \\ &= \ldots \\ &= 3^{n-1} x(1) = 4 \cdot 3^{n-1}. \end{aligned}$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$x(n) = 3x(n-1) \quad \text{for } n > 1, \quad x(1) = 4$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] = 3^2 x(n-2) \\ &= 3^2[3x(n-3)] = 3^3 x(n-3) \\ &= \ldots \\ &= 3^i x(n-i) \\ &= \ldots \\ &= 3^{n-1} x(1) = 4 \cdot 3^{n-1}. \end{aligned}$$

Note: The solution can also be obtained by using the formula for the n term of the geometric progression:

$$x(n) = x(1)q^{n-1} = 4 \cdot 3^{n-1}.$$

Question # 06

Solve the following recurrence relations

x (n) = 3x (n-1)

for n > 1, x(1) = 4

x (n) = x(n/2) + n

for n > 1, x(1) = 1

$$
\begin{aligned}
&x(n) = x(n/2) + n \quad \text{for } n > 1, \quad x(1) = 1 \quad (\text{solve for } n = 2^k)\\
x(2^k) &= x(2^{k-1}) + 2^k\\
&= [x(2^{k-2}) + 2^{k-1}] + 2^k = x(2^{k-2}) + 2^{k-1} + 2^k\\
&= [x(2^{k-3}) + 2^{k-2}] + 2^{k-1} + 2^k = x(2^{k-3}) + 2^{k-2} + 2^{k-1} + 2^k\\
&= \ldots\\
&= x(2^{k-i}) + 2^{k-i+1} + 2^{k-i+2} + \cdots + 2^k\\
&= \ldots\\
&= x(2^{k-k}) + 2^1 + 2^2 + \cdots + 2^k = 1 + 2^1 + 2^2 + \cdots + 2^k\\
&= 2^{k+1} - 1 = 2 \cdot 2^k - 1 = 2n - 1.
\end{aligned}
$$